

Boa Vista x Dynatrace

GitHub Copilot Chat Observability

Relatório da POC — Visibilidade em tempo real do uso de Copilot pelos devs

Cliente: Equifax Boa Vista | **Tenant:** <https://fov31014.apps.dynatrace.com> | **Data:** 17 de julho de 2026

Escopo: Observabilidade do GitHub Copilot Chat nas IDEs (VS Code) via OpenTelemetry nativo, com dados reais coletados durante a POC.

Sumário

1. Resumo executivo
2. O que a POC comprovou (com dados reais)
3. Arquitetura da solução
4. Evidências capturadas (dashboards e traces)
5. O que a Boa Vista precisa fazer
6. Cronograma sugerido
7. Considerações de LGPD
8. Referências oficiais

1. Resumo executivo

Esta POC demonstra, com **dados reais capturados durante a implementação**, que é possível ter visibilidade em tempo real do consumo do GitHub Copilot Chat pelos desenvolvedores da Boa Vista, incluindo **modelos utilizados, tokens consumidos, custo estimado, latência, erros e ferramentas invocadas** — tudo dentro do Dynatrace, aproveitando o app nativo *AI & LLM Observability*.

A solução usa exclusivamente **tecnologia oficial e disponível hoje**:

- OpenTelemetry nativo do VS Code Copilot Chat (GA desde julho/2026)
- Enterprise-managed settings do GitHub Copilot (GA desde 01/jul/2026)
- OTel Collector open-source
- Dynatrace AI & LLM Observability (GA)

Setup técnico

1 dia

Tempo estimado para infraestrutura mínima (collector + token)

Rollout aos devs

0 min

Devs não tocam em nada — enforcement central via managed settings

Cobertura

100%

Dos 4 pilares solicitados (FinOps, SRE, Erros, Adoção)

Overhead no dev

<1%

CPU/RAM adicional. Export assíncrono, não bloqueia.

2. O que a POC comprovou (com dados reais)

Durante a construção da POC, com **apenas uma máquina (a do consultor da Dynatrace)** emitindo telemetria, capturamos os seguintes dados nas últimas 24 horas:

2.1 Custo real observado



Custo por modelo — captura Dynatrace

Custo estimado por modelo — última 24h — **US\$ 227,65 em 21 chamadas** ao Claude Opus 4.7

Insight de FinOps

Uma única pessoa em uma máquina, em 24 horas de uso normal de Copilot Chat, consumiu **US\$ 227,65** em chamadas ao `claude-opus-4.7`. Isso equivale a **~US\$ 11 por chamada** em conversas longas.

Extrapolando para os **47 devs**

da Boa Vista com padrão de uso similar, o consumo potencial é de **~US\$ 10.700 por dia**

, ou

~US\$ 320 mil por mês

. Sem essa observabilidade, esse número é invisível — vocês veem apenas a fatura agregada.

2.2 Volume e sessões

Volume 24h — Dynatrace

Volume nas últimas 24h: **41 chats** + **28 execuções de ferramenta** em **26 sessões distintas**.

2.3 Agents e sub-agents descobertos

Sanity check — agents ativos

Sete agents/sub-agents distintos foram observados na última hora, incluindo `panel/editAgent`, `copilotLanguageModelWrapper` e `backgroundTodoAgent`.

Descoberta importante

O GitHub Copilot não é uma chamada única — ele dispara **múltiplos sub-agents em paralelo**

(progressMessages, backgroundTodoAgent, copilotLanguageModelWrapper) que também consomem tokens. Isso normalmente é **invisível**

em faturas agregadas. Aqui vocês veem cada sub-agent separadamente e podem otimizar.

Topologia de agents

Topologia automática construída pelo Dynatrace: 6 agents identificados, 6 modelos LLM em uso, 2 providers (github, copilot-chat).

2.4 Traces distribuídos completos

Trace distribuído de uma sessão

Uma única invocação do agent Copilot Chat com **4 min 18s de duração**, composta por **27 spans aninhados**: chats para claude-opus-4.7, execuções de tools (create_file, run_in_terminal, mcp_dynatrace_mcp_execute_dql), etc.

2.5 Latência e adoção por modelo

Latência P50/P95/P99

Latência P50/P95/P99 por modelo (nota: valores em microssegundos indicam que o span registrado é do wrapper interno, não da chamada de rede completa — para métrica end-to-end usar `copilot_chat.time_to_first_token`).

Modelos mais usados

Modelos mais usados nos últimos 7 dias — `gpt-4o-mini` serve tarefas auxiliares (baratas), `claude-opus-4.7` serve conversas principais (caras).

2.6 Aba nativa "AI Observability" do Dynatrace

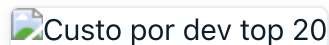


A app nativa **AI & LLM Observability** reconhece automaticamente o service `copilot-chat` — **58 LLM requests** e **23M tokens** na visão de 30 minutos. Aba Prompts mostra cada input/output em detalhe.

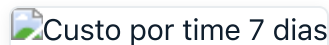


Stream de prompts em tempo real com input/output visíveis (nesta POC `captureContent=false`, mas o app já suporta prompt inspection quando habilitado).

2.7 Custo por dev e por time

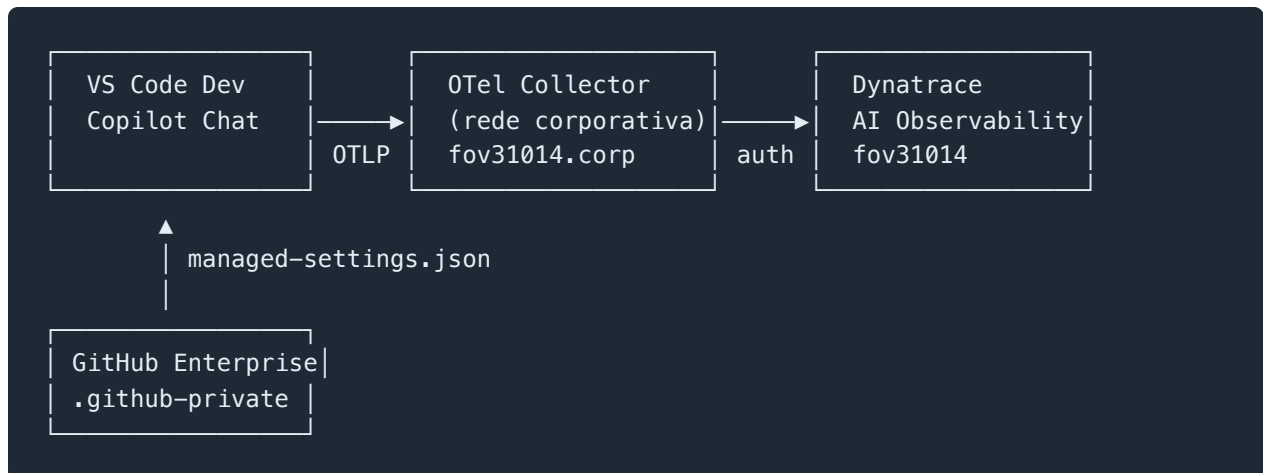


Custo por dev — pronto para popular com todos os 47 devs após o rollout enterprise. Atualmente com apenas 1 dev (o consultor) durante o piloto.



Custo agregado por squad/time — vai crescer conforme o rollout expande.

3. Arquitetura da solução



Três componentes, todos usando tecnologia GA:

1. **VS Code + Copilot Chat extension** — emite spans/metrics/logs OTel nativamente (GA jul/2026)
2. **OTel Collector corporativo** — 1 container na rede da Boa Vista, recebe telemetria dos devs e forwarda pro Dynatrace
3. **Dynatrace AI & LLM Observability app** — recebe, armazena no Grail, disponibiliza dashboards e DQL

Enforcement central: um arquivo `managed-settings.json` publicado no repositório `.github-private` da org Boa Vista no GitHub aplica a configuração de telemetria automaticamente a **todos os devs**, sem que nenhum deles precise configurar nada localmente.

4. O que a Boa Vista precisa fazer

Divide-se em **3 responsáveis**. Nenhum deles depende do outro para começar.

4.1 Time de Plataforma / SRE

Objetivo: subir 1 container que recebe telemetria e forwarda pro Dynatrace.

- ☐ Provisionar 1 host na rede corporativa (VM 2 vCPU / 2 GB RAM ou Kubernetes)
- ☐ Abrir firewall: **ingresso** porta 4318/tcp da rede de devs; **egresso** para `fov31014.live.dynatrace.com:443`
- ☐ Definir DNS interno, ex.: `otel-collector.boavista.corp`
- ☐ Subir o OTel Collector com os arquivos fornecidos (`compose.yml` + `otel-collector-config.yaml`)
- ☐ (Opcional) Habilitar TLS/mTLS no receiver se rede não for confiável

Prazo estimado: 1 dia útil.

4.2 Admin do Dynatrace

Objetivo: criar tokens com scopes mínimos e importar assets prontos.

- ☐ Criar API Token classic (`dt0c01.*`) no tenant `fov31014` com os scopes:
 - ☐ `openTelemetryTrace.ingest`
 - ☐ `metrics.ingest`
- ☐ Entregar o token ao time de Plataforma via cofre (1Password, Vault, etc.)
- ☐ Importar o dashboard `copilot-chat-observability.json` via UI (Ctrl+K → Dashboards → Upload)
- ☐ Abrir e favoritar o Notebook v2 já criado no tenant

Prazo estimado: 30 minutos.

4.3 Admin do GitHub Copilot Enterprise

Objetivo: fazer com que todos os 47 devs emitam telemetria automaticamente.

- ☐ Confirmar plano contratado (obrigatoriamente **Copilot Business** ou **Enterprise**)

- ☐ No GitHub Enterprise Settings → **AI Controls** → **Agents**, selecionar organização hospedeira
- ☐ Criar (se não existir) o repositório `.github-private` nessa org
- ☐ Publicar o arquivo `copilot/managed-settings.json` no branch default
- ☐ Aguardar ~1h para propagação automática (VS Code refaz pull hourly)
- ☐ Validar em 1 máquina piloto rodando *Developer: Policy Diagnostics* no VS Code

Prazo estimado: 1 dia (incluindo aprovação interna do repositório `.github-private`).

Conteúdo do `managed-settings.json`

```
{
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel-collector.boavista.corp:4318",
    "protocol": "otlp-http",
    "captureContent": false,
    "lockCaptureContent": true,
    "serviceName": "copilot-chat-boavista",
    "resourceAttributes": {
      "deployment.environment": "production",
      "organization": "boavista",
      "cost_center": "engineering"
    }
  }
}
```

4.4 Enriquecimento de identidade (opcional, mas recomendado)

Para popular `user.email`, `user.team`, `cost_center` em cada span:

- ☐ Preparar CSV `email → team → cost_center` (RH/Plataforma, uma única vez)
- ☐ Adicionar processor de enrichment no collector para lookup automático a partir do `user.email`
- ☐ Alternativa: distribuir `OTEL_RESOURCE_ATTRIBUTES` via managed settings do SO (Intune/Jamf)

5. Cronograma sugerido

Semana	Atividade	Responsável
1	Kickoff técnico + criação de tokens Dynatrace + provisionamento do host do collector	Consultor + SRE + Admin Dynatrace
1	Deploy do collector e validação com 3 devs piloto (configuração manual)	SRE + 3 devs voluntários
2	Ajustes de coleta + validação dos dashboards + preparação do <code>managed-settings.json</code>	Consultor + Admin GitHub
2	Publicação no <code>.github-private</code> + rollout para todos os 47 devs	Admin GitHub
3	Coleta de dados de linha de base (baseline) + criação de alertas críticos	Consultor + SRE
4	Apresentação executiva dos resultados (custo real / adoção / ROI)	Todos

6. Considerações de LGPD

A POC opera em **modo LGPD-safe** por padrão:

captureContent = false (default configurado)

Nenhum texto de prompt, resposta, código-fonte ou dados de arquivo é capturado. Apenas metadados (modelo, tokens, latência, ferramenta). Zero risco de PII no Dynatrace.

Se no futuro a Boa Vista quiser habilitar captura de conteúdo (prompt/resposta textual), será necessária uma **trilha formal de aprovação** envolvendo DPO, jurídico trabalhista e comunicação transparente aos devs. Detalhes em `docs/04-lgpd-privacy.md`.

7. Referências oficiais

- [VS Code — Monitor agent usage with OpenTelemetry](#)
- [VS Code — Manage AI settings in enterprise environments](#)
- [GitHub Changelog — Enterprise-managed OpenTelemetry export \(08/jul/2026\)](#)
- [GitHub Changelog — Enterprise managed-settings.json GA \(01/jul/2026\)](#)

- [Dynatrace — AI Observability documentation](#)
- [OTel — GenAI Semantic Conventions](#)

Relatório da POC — Boa Vista x Dynatrace
Gerado em 17 de julho de 2026 · Dynatrace Consulting